CE152: Object-Oriented Programming

Data types

Dr. Sebastian Halder

CSEE

2023

Refresher on data types

Primitives and Objects

- In Java there are two types of variables: *primitives* and *Objects*
- *Primitives* are the basic building blocks of classes
- *Classes* are made of these building blocks or other *classes*
- *Objects* are instances of classes
- Numerous *Classes* are built into the Java Class Library (JCL)
- ***VERY IMPORTANT:*** In Java most lines end with a semicolon and you *must declare the type* of the variable!
- Of course there are exceptions but basically all declarations and expressions end with a semicolon

```
// By the way, this is how you declare a comment in Java
int i = 5; // primitive integer
Integer j = new Integer(10); // Object integer
j.toString(); // Objects can have methods
              // They are based on a class!
```

Default values

- The default value of primitives is `0`
- Or the corresponding representation
  - `float` 0.0f
  - `boolean` false
  - `char` \u0000
- Wrapper classes (Integer, Double ...) are initialised to `null`
- *It is good practice to initialise variables!*
- Sometimes this is not possible though and then we need to assign values after creating the variable

Primitives are more memory efficient

```
/* You may now be wondering why not always use Objects?
The primitive data types are much more memory efficient!
So use them when you can.

This by the way is how you declare a multiline
comment in Java. */
int x = 1; // this uses 32 bits...

Integer y = new Integer(1); // this uses 128 bits!
```

Size of Java types

Primitive	Size	Wrapper	Size
`boolean`	8-32 bits	`Boolean`	128 bits
`byte`	8 bit	`Byte`	128 bits
`short, char`	16 bits	`Short, Character`	128 bits
`int, float`	32 bits	`Integer, Float`	128 bits
`long, double`	64 bits	`Long, Double`	192 bits

- `boolean` is encoded as an `int` (32 bit) when used as a single variable and as a `byte` (8 bit) when used in arrays
- The amount of space a variable or array takes in the JVM may additionally vary
- We should aim to use the smallest possible data type for efficiency and readability

This video is about the variables

```
class MyClass{
  /*
  1. List variables
  */
  int x;
  boolean b;
  String s;

  /*
  2. Define constructor
  */

  /*
  3. Add methods
  */
}
```

Car example

```
class Car{
  /*
  1. List variables
  */
  int speed;
  String colour;

  /*
  2. Define constructor
  */

  /*
  3. Add methods
  */
}
```

Primitives

List of primitives

Type	Size (bits)	Minimum	Maximum
`byte`	8	-2^7	$2^7 - 1$
`short`	16	-2^{15}	$2^{15} - 1$
`int`	32	-2^{31}	$2^{31} - 1$
`long`	64	-2^{63}	$2^{63} - 1$
`float`	32	-2^{-149}	$2 - (2^-23) * 2^{127}$
`double`	64	-2^{-1074}	$(2 - 2^{-52}) * 2^{1023}$
`char`	16	0	$2^{16} - 1$
`boolean`	1	–	–

Examples

```
byte b = 10;
short s = 20_000;
int i = 300_000_000;
long l = 400_000_000_000_000L;
float f = 5.123f;
double d = 6.456789012345678;
char c = 's';
boolean bb = true;
```

- We can use _ to make longer numbers easier to read
- The _ has no effect on the value as such

Remark on floating point numbers (1)

- As you saw in the table the definition of minimum of maximum for `float` and `double` is a bit complicated
- You do not need to know how exactly floating point numbers are defined for this module
- You may look at the definition if you are interested though: https://en.wikipedia.org/wiki/IEEE_754-1985

Remark on floating point numbers (2)

- What is important to know that all data type of the same size are able to represent the same number of values
- E.g. `int` can represent as many values as `float`
- What differs is the range and the size of the intervals
- The integer data types all represent all integers in their range
- The floating point data types represent the numbers in their range with a certain precision
- Interestingly the precision varies with the number
- Some integers can actually not be represented by `float` (e.g.: 16_777_217) or `double` (e.g.: 9_007_199_254_740_993)
- If we assign them to `float`/`double` the value will be rounded to the next possible value

Why do `byte`, `short`, `int` and `long` exist?

- In theory we could use `long` anytime we want to store integers to be on the safe side
- It is *good practice* to always use the data type that covers the expected data range
- The impact on performance is very small though with the number of variables we will be dealing with in CE152
- The information that you provide on what you expect the data type to contain may be the most important aspect
- ***Remember***: you do not only write code to achieve a particular goal but to also share it with other people which makes ***readability*** very important
- The data types `byte` and `short` are more important for embedded systems
- The same is true for `float` and `double`

Usage

- We will look at some more examples during the live coding

Objects

Objects and classes

- In Object-Oriented Programming we create building blocks
- These building blocks consist of ***templates or blueprints: the classes***
- When we instantiate classes we create ***objects*** using the ***constructor*** of that class
- This is analogous to objects in reality where we have classes, e.g. chair, and can have multiple objects of that type
- Please refer to the video on *Objects* for further information
- In the following we will look at the built-in objects provided by the JDK

Wrappers

- The primitives we discussed earlier all of have analogous wrapper classes
- These are Objects containing the primitive
- The reason for the existence of wrapper classes is that many principles we use in OOP can only be applied to Objects
- For example, there are very useful classes to store multible variables, so-called collections, that can only contain Objects
- The reason that they contain only Objects is that all variables contained in them must implement certain methods, e.g. for sorting
- ***Please remember: Only Objects have methods***
- ***Please remember: Primitives are not Objects, Wrappers are***

Instantiating wrapper Objects

- We can use the `new` operator or methods dedicated for creating new Wrapper Objects from primitives

```
Integer wrapperOne = new Integer(5);
Integer wrapperTwo = Integer.valueOf(5);
```

- We can convert the wrapper back to a primitive

```
int i = wrapperOne.intValue();
```

Use of Wrappers

- We can determine the minimum and maximum values of the primitives used variables contained in the wrapper classes:

```
Double.TYPE; Double.SIZE;
Double.MIN_VALUE; Double.MAX_VALUE;
Integer.TYPE; Integer.SIZE;
Integer.MIN_VALUE; Integer.MAX_VALUE;
```

- We can use Wrappers to convert text to numerical data (this is very useful when reading data from files):

```
int i = Integer.parseInt("152");
double d = Double.parseDouble("152.152");
```

- As mentioned before Wrappers allow the use of Java Collections (more on that in another lesson)!

Important classes for CE152

- There are about 4500 classes in the JDK
- For now we will look at some useful examples: `String` (and related classes), `System`, `Math` and `Object`
- `String` based operations are quite common and something that is easily possible using primitives
- Essentially `String` is an class that can hold multiple (or just one) `char`

Examples of creating a String

```
String s1 = String.valueOf('c');
String s2 = new String('c');
String s3 = new String("CE152");
String s4 = new String('CE152'); //NOT POSSIBLE!!
```

Using String

```
String s1 = String.valueOf("abcde");
char c1 = s1.charAt(0); // a; indices start with 0
char c2 = s1.charAt(2); // c
s1 = s1.toUpper(); //ABCDE
s1 = s1.toLower(); //abcde
int i = s1.length(); //5
String s2 = String.valueOf("ABCDE");
boolean b1 = s1.equals(s2); // false
boolean b2 = s1.equalsIgnoreCase(s2) //true
```

StringBuilder

- Sometimes we want to construct a `String` in steps
- For this we can use `StringBuilder` (this is another class!)

```
StringBuilder sb = new StringBuilder();
sb.append('a');
sb.append('b');
sb.append('c');
String s = sb.toString();
```

- We use `StringBuilder` becauser operations like this actually create a `StringBuilder` in the background in each step which is slower

```
String s = new String();
s += 'a';
s += 'b';
s += 'c';
```

Printing to the command line

- In Java we use a class called `System` for printing to the command line
- This class has a class variable called `out` of type `PrintStream` which is `static` (this means we can just use it without assigning it to a variable)
- This variable has a method called `println()`
- `System` has some other variables has well that we will learn about later
- This method is what we use to print in Java

```
System.out.println("Hello CE152");
```

Mathematical operations

- Another class that will be useful in this module is `Math`
- Similar to `System` we will use `static` methods and variables

```
Math.round(1.234); // rounds to nearest int
double pi = Math.PI; // returns PI
int x = 5; int y = 10;
Math.max(x, y); // returns 10
Math.min(x, y); // returns 5
/* And many other functions:
  sin, cos, log, floor, ceiling...
  Have a look using the IDE!
*/
```

Object

- `Object` is a class in Java
- The classes in Java are arranged in a tree and connected via *inheritance* which you will learn about in one of the next videos
- `Object` is the root node of this tree which means all other classes are related to `Object`
- `Object` has to very important methods: `toString()` and `equals()`
- `toString()` converts to owning `Object` to `String` allowing us to print it (e.g. using `System.out.println()`)
- `equals()` compares the `Object` to another and returns a `true` if they are equal
- Since all classes in Java are derived from `Object` they all have these two methods (more about that in the video on inheritance)

Java Library and Packages

- Java comes with many useful libraries
  - This is often called the API: *advanced programmer interface*
  - see https://docs.oracle.com/en/java/javase/index.html
- As we have seen the library classes are organised in Packages
  - `java.lang`, `java.util`, `java.net`, `java.util.regex`, `javax.swing`...
- Everything in `java.lang` is automatically known to the compiler
  - e.g. `String` would otherwise be `java.lang.String`
- We can use the keyword `package` to organise our own code in this way

Import statements

- The `import` statements make classes accessible without having to write the full names
- Must be placed before any class declarations
- With the star symbol we can import all classes from one package
  - If this leads to a name clash we have to import individual classes instead
- We can import our packages to other packages in the same way

Importing packages (examples)

```
import java.io.File;
import java.io.FileNotFoundException;
import java.io.PrintStream;
import java.util.Arrays;
import java.util.Scanner;
```

Creating packages in IntelliJ

- In the Project View, select the “src” folder
- Then click on “File -> New -> Package” and create a new package called e.g. lab1
- Select the “lab1” package in the project view, then click on “File -> New -> Classes” and create new classes
- This class is created in then package lab1

Operators

Arithmetic operators

- We use arithmetic expression in the same way as in algebra

Operator	Meaning	Operation	Result
+	addition	1 + 2	3
-	subtraction	7 - 2	5
*	multiplication	2 * 3	6
/	division	10 / 2	5
%	modulus	11 % 2	1
++	increment	2++	3
--	decrement	9--	8

Comparison operators

- We will use these in loops (`for`, `while`) and conditionals (`if`, `else`...)

Operator	Meaning	true	false
`==`	equal	`4 == 4`	`4 == 5`
`!=`	unequal	`4 != 5`	`4 != 4`
`<`	less than	`4 < 5`	`4 < 4`
`<=`	less than or equal	`4 <= 5`	`5 <= 4`
`>`	greater than	`5 > 4`	`5 > 5`
`>=`	greater than or equal	`5 >= 5`	`4 >= 5`

Logical operators

- If we wanted to have two conditions in a loop we could combine both statements with a logical operator:
  - Logical *and*: &&
  - Logical *or*: ||
  - Logical *not*: !
- You should know how these work from the previous term
- Please revise this if you do not!

Assignment operators

Operator	Meaning	Operation	Result
=	assigns values from right to left side	`x = 1 + 1`	x will be 2
+=	assigns sum of left and right to left	`x = 2; x += 1;`	x will be 3
-=	assigns difference of left and right to left	`x = 3; x -= 1;`	x will be 2
*=	assigns product of left and right to left	`x = 4; x *= 2;`	x will be 8
/=	assigns quotient of left and right to left	`x = 10; x /= 2;`	x will be 5

- There are some more but we will not be using them in CE152

Remarks on concatenation

- `String` can be concatenated with another `String`
  - `"AB" + "C"` becomes `"ABC"`
- `String` can be concatenated with `int`
- It is important to note that **operator precedence** for + is from left to right
- How will this affect the output of the following code?

```
String s1 = "40 + 2 = " + 40 + 2;
String s2 = "40 + 2 = " + (40 + 2);
String s3 = 40 + 2 + " = " + 40 + 2;
```

Casting

Changing the data type

- We heard about casting e.g. when we cast a char to an int and vice versa:

```
// Will the variable b contain a letter or a number?
int a = (int)'b';
char b = (char)a;
```

- We will learn some rules for casting Objects in the video about inheritance!

Casting primitives

- The types of casting that are distinguished in this case are *widening* and *narrowing* casting
- Remember an int had 32 bits and a double had 64 bits?
- If we cast from *int to double* it is widening and when we cast from *double to int* it is narrowing casting

Widening vs. narrowing

```
int a = 127;
// This is fine
double b = a;
// This will produce an error (compilation)
int c = b;
// This is fine
int d = (int)b;
```

- A widening cast is automatic, a narrowing cast must be declared
- The reason behind these rules is that casting from a double to an int will cause us to lose information
- The compiler wants to make sure we know this risk (by the way char has 2 bytes (16 bits) so we are narrowing when casting from int to char)

Live coding

- Example use of all data types!
- Casting!
- Creating projects and packages!
- Operators!

Summary

Java documentation

- JRE8: https://docs.oracle.com/javase/8/docs/
- You can find information on all data types in the API documentation: https://docs.oracle.com/javase/8/docs/api/
- JRE11: https://docs.oracle.com/en/java/javase/11/
- Java 11 API: https://docs.oracle.com/en/java/javase/11/docs/api/index.html

Thank you and see you in the next video!